

ADVANCED DATA ANALYSIS WITH PYTHON

Session 1: Kickoff & Python Environment

Course overview, grading, and getting your setup right

Silyia Tulli
Cecilia Graiff

About us



Silvia Tulli

Associate Professor, Sorbonne University
Head of AI Research, HABS

Research focus: AI and robotics, multimodal machine learning.
Teaches sessions on Python environment & tooling, functional programming, data wrangling, and reproducible research.



Cecilia Graiff

Ph.D. Student, ALMAAnaCH, Inria Paris

Research focus: AI, NLP, machine learning. Teaches sessions on control flow & data structures, OOP, web scraping, text analysis and NLP, and geospatial analysis.

Today's agenda

01

Welcome & introductions

Who we are, how the course works

02

Course overview

Description, prerequisites, expected skills, format

03

Grading

How the course is evaluated, and what the final project involves

04

Detailed syllabus

All 12 sessions, topic by topic

05

Election of class representative

Your point of contact with us throughout the semester

06

Technical session: Python environment & AI coding tools

Coding style, virtual environments, pip, and AI-assisted coding

Wooclap Kickoff

- Event Code: SUGNNGF



Advanced Data Analysis with Python

Building on core Python and pandas skills, this course moves into statistical modeling and machine learning applied to real-world data

- Work with real-world datasets provided in near-clean form — light cleaning, focus on analysis
- **Learn the theory and application of** linear regression, regularized regression, logistic regression, classification trees, and time series forecasting
- Build a full machine learning pipeline: data splitting, model fitting, and evaluation
- Practice critically analyzing and reporting on experimental results

Collect

Clean

Explore

Model

Evaluate

Prerequisites and expected skills

COMING IN, YOU SHOULD KNOW

- Python core concepts: data types, control structures, functions, error handling
- Data handling with pandas: opening, cleaning, and manipulating datasets; basic numpy
- Basic statistics: summary statistics, simple linear regression

BY THE END, YOU'LL BE ABLE TO

- Apply ML and statistics theory in Python: regression, classification, time series
- Draft and run a full data analysis pipeline end to end
- Collect, clean, explore, visualize, model, and evaluate real data

Wooclap Prerequisites

- Event Code: MGQYYOF



Python core concepts

- `7 // 2` → `int` — Floor division (`//`) between two integers always returns an `int` in Python (it discards the remainder rather than producing a decimal).
- `range(2, 8, 2)` → `2 4 6` — `range(start, stop, step)` begins at 2, stops *before* reaching 8, and increments by 2 each time: 2, then 4, then 6 (8 is excluded).
- **No `return` statement** → `None` — If a function reaches its end without hitting a `return`, Python implicitly returns `None`; it doesn't raise an error or default to 0.
- **Mutable structure** → `list` — Lists can be changed in place (append, remove, modify elements) after creation. Tuples, strings, and frozensets are all immutable.

Pandas and numpy

- **Load a CSV** → `pd.read_csv()` — This is the standard pandas function for reading a CSV file into a DataFrame. `pd.load_csv()`, `pd.open_csv()`, and `pd.import_csv()` don't exist.
- **Remove missing values** → `df.dropna()` — `dropna()` removes rows (or columns) containing NaN values. `fillna()` does the opposite — it fills them in rather than removing them.
- **Filter rows where age > 30** → `df[df['age'] > 30]` — This is pandas' boolean indexing pattern: `df['age'] > 30` creates a True/False mask, and `df[mask]` keeps only the rows where it's True.
- `np.array([1, 2, 3, 4]).shape` → `(4,)` — A one-dimensional array of length 4 has shape `(4,)` — a single-element tuple, not `(1, 4)` or `(4, 1)`, which describe 2D arrays.
- **Rename columns** → `df.rename(columns=...)` — This is the correct method signature for relabeling one or more columns, e.g. `df.rename(columns={'old': 'new'})`.

Basic statistics

- **Typical value with outliers** → **Median** — The median (the middle value when sorted) is resistant to extreme outliers, unlike the mean, which outliers can pull sharply up or down.
- **Standard deviation measures** → **How spread out the values are** — It quantifies the dispersion of data points around the mean; it says nothing about the average or most frequent value directly.
- In $y = a + bx$, **b** is → **The slope** — **b** represents how much **y** changes for each one-unit increase in **x**. **a** is the intercept (the value of **y** when **x** = 0).
- $R^2 = 0.85$ means → **85% of the variance in y is explained by the model** — This is the precise definition of R^2 . It is *not* a claim about prediction accuracy on new data, nor about correlation strength or outlier proportion directly.
- **Correlation near 0** → **Little to no linear relationship** — A coefficient close to 0 means the two variables show little to no *linear* association (it says nothing about causation, and doesn't imply a strong relationship of either direction).

Academic format and materials

- **Materials are shared on Moodle and GitHub** — slides and Jupyter notebooks
- Each session opens with a short theoretical part, then moves into hands-on coding in notebooks
- **Expect a mix of** individual exercises, small-group work, and live Wooclap quizzes

<https://github.com/cg67qowo/Advanced-Python>
<https://moodle.sciences-po.fr/course/view.php?id=64748>

2h

in class, every week

12

sessions this semester

~2h50

expected work per week outside class

How the course is graded



20% — Continuous assessment. Short in-class quizzes (Wooclap) throughout the semester, checking understanding session by session.

80% — Final project. Submitted at the end of the course; evaluated on both the practical output and a written report.

Validation mode: the final project

- **Content** — a data analysis project applying the methods covered in the main course (regression, classification, or time series), on a topic agreed together with the professors during the semester
- **Due** — submitted at the end of the course (exact date confirmed via Moodle)
- **Criteria** — practical output (code, models, data visualizations) and a final written report
- **Format** — details on individual vs. group work to be confirmed as the topic is set

WHAT TO KEEP IN MIND

The project topic connects directly to what you're studying in your main policy/analysis courses — **start thinking early about a dataset or question you care about.**

Detailed syllabus — 12 sessions

01	Python Environment & Best Practices, AI-Assisted Coding Tools	Silvia Tulli
02	Advanced Control Flow and Data Structures	Cecilia Graiff
03	Advanced Functions and Functional Programming	Silvia Tulli
04	Object-Oriented Programming and Modules	Cecilia Graiff
05	Recap	Silvia Tulli
06	Web Scraping	Cecilia Graiff
07	Advanced Data Wrangling	Silvia Tulli
08	Text Analysis and NLP	Cecilia Graiff
09	Statistical Analysis and Machine Learning Basics	Silvia Tulli
10	Geospatial Analysis and Advanced Visualisation	Cecilia Graiff
11	Reproducible Research, GitHub and Hugging Face	Silvia Tulli
12	Recap	Cecilia Graiff

Why we're electing a class representative

- Acts as the main point of contact between the class and us throughout the semester
- We'll turn to them for quick, informal checks — e.g. a poll on pacing, or scheduling group work
- **They gather and relay class feedback at the mid-semester point**, so we can adjust before it's too late
- It's a light commitment: mostly relaying messages both ways

CLASS REPRESENTATIVE

Let's elect one now



Shuyu



NOW, PART TWO

Python Environment & AI-Assisted Coding Tools

Before we get into the semester's material, let's make sure everyone's setup is solid: writing readable code, isolating dependencies, and knowing what AI tools can (and can't) help with.

Photo: Near
Rifugio Agostini, Dolomites,
Italy

Python environment & style

01

Intermezzo: Coding Style

The Python tutorial's own note on PEP 8 and readable code

docs.python.org/3/tutorial/controlflow.html#intermezzo-coding-style

02

Creating Virtual Environments

Why and how to isolate a project's dependencies with venv

docs.python.org/3/tutorial/venv.html#creating-virtual-environments

03

Managing Packages with pip

Installing, listing, freezing, and reproducing dependencies

docs.python.org/3/tutorial/venv.html#managing-packages-with-pip

Coding style: why it matters

- Code is read far more often than it is written — optimize for the reader
- **Follow PEP 8:** 4-space indentation, meaningful names, spaces around operators
- Use blank lines to separate logical sections, not to decorate
- Consistency inside a file matters more than any single rule
- Docstrings and comments explain why, not just what

```
# readable, PEP 8 style
def average(values):
    # guard against empty input
    if not values:
        return 0
    return sum(values) / len(values)
```

Virtual environments

- A **virtual environment** is a self-contained Python install with its own packages
- Keeps project dependencies isolated — no version conflicts between projects
- Created once per project, activated every time you work on it
- Never install project packages into your system-wide Python

```
# create the environment
python -m venv .venv

# activate it (macOS/Linux)
source .venv/bin/activate

# activate it (Windows)
.venv\Scripts\activate
```

Managing packages with pip

- pip installs packages into the active environment
- pip freeze captures exact versions for reproducibility
- **A requirements.txt file** lets collaborators recreate your setup
- Always activate the environment first — check with which python

```
pip install requests pandas

# save exact versions
pip freeze > requirements.txt

# recreate elsewhere
pip install -r requirements.txt
```

The current landscape

Four tools worth knowing as a Python student — each integrates AI assistance differently into the way you write code.

Codex	OpenAI's coding agent, available via CLI, IDE extension, and ChatGPT — reads a codebase, plans changes, and executes them
Copilot	GitHub's assistant, built into VS Code, JetBrains, and the CLI, with inline suggestions and chat
JetBrains AI	Native assistant inside PyCharm/IntelliJ — context-aware completion, refactoring, and chat tied to the IDE
Claude	Anthropic's assistant, usable in chat, the API, or as a coding agent (Claude Code) for larger tasks



The current landscape

OpenAI Codex

- **An agentic coding tool:** give it a task, it reads relevant files, edits code, and can run commands
- Available as a terminal CLI, an IDE extension, and inside ChatGPT
- Well suited to multi-step tasks: refactors, bug fixes, or scaffolding a new script
- Still needs a human to review the diff before trusting it — treat it as a fast first draft

GitHub Copilot

1

Getting started

Sign in with GitHub, install the extension for your editor, and suggestions appear as you type.

2

In notebooks & the CLI

Copilot also assists inside Jupyter notebooks and offers command explanations at the terminal.

3

Free for students

Verified students get Copilot free through the GitHub Student Developer Pack.

Claude as a coding assistant

- Usable directly in chat for explaining, writing, or debugging Python
- Available via the API for building tools and automations
- **Claude Code** extends this to an agentic mode — planning and executing multi-file changes
- Strong at explaining why code works, which is useful while you're still learning

Verifying AI-generated code

- Never paste and run AI output without reading it first — you're accountable for what it does
- Check the logic against the actual assignment requirements, not just “does it run”
- Test edge cases yourself: empty input, wrong types, boundary values
- Ask the assistant to explain unfamiliar lines rather than accepting them blindly
- Suggestions can be outdated or subtly wrong — treat them as a draft, not an answer key

Academic integrity & AI tools

- Follow your course and institution's stated policy on AI use — it varies by assignment:

sciencespo.fr/en/news/artificial-intelligence-at-sciences-po/

- When in doubt, disclose how a tool helped rather than guessing at the rule
- **Using AI to understand** a concept differs from using it to produce your submission unedited
- The goal of this course is your understanding — AI assistance should support that, not replace it

AI should be "a lever for deploying skills, not a substitute for critical thinking."

See Sciences Po's guidelines: sciencespo.fr

Academic integrity & AI tools

- Follow your course and institution's stated policy on AI use — it varies by assignment:

sciencespo.fr/en/news/artificial-intelligence-at-sciences-po/

- When in doubt, disclose how a tool helped rather than guessing at the rule
- **Using AI to understand** a concept differs from using it to produce your submission unedited
- The goal of this course is your understanding — AI assistance should support that, not replace it

Where AI use is permitted, students must document it

See Sciences Po's guidelines: sciencespo.fr

Academic integrity & AI tools

- Follow your course and institution's stated policy on AI use — it varies by assignment:

sciencespo.fr/students/en/study/academic-integrity/

- When in doubt, disclose how a tool helped rather than guessing at the rule
- **Using AI to understand** a concept differs from using it to produce your submission unedited
- The goal of this course is your understanding — AI assistance should support that, not replace it

[How-to sheet on using
generative AI at Sciences Po](#)

See Sciences Po's guidelines: sciencespo.fr

WRAP-UP

Before next session

- Create a virtual environment for your project directory
- Install one AI coding tool and try it on a small script
- Re-read the coding style guide with your own recent code in mind
- Reach out to your class representative with any early questions